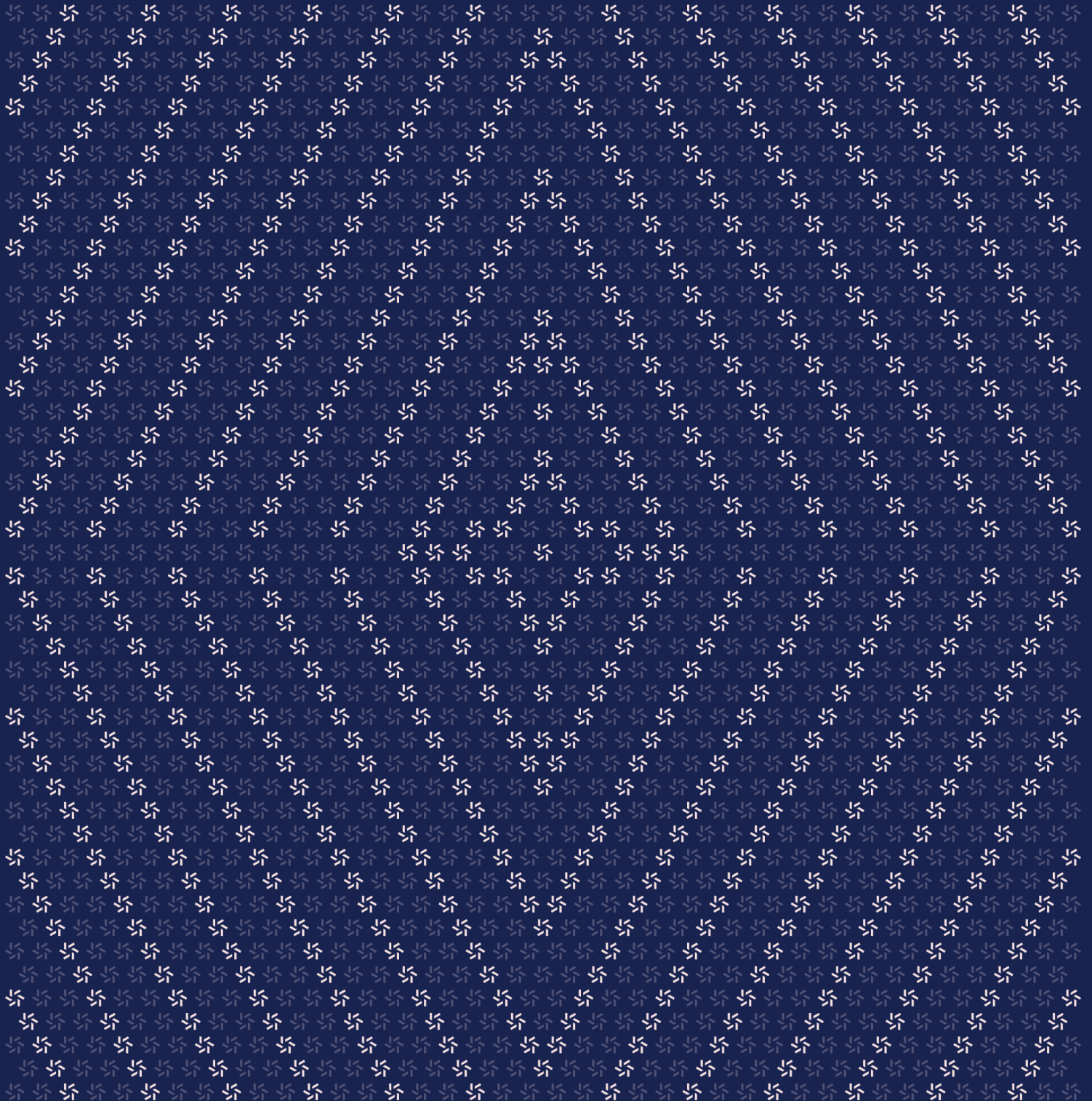


June 11, 2026

Lighter (EVM)

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	6
2. Introduction	7
2.1. About Lighter (EVM)	7
2.2. Methodology	7
2.3. Scope	9
2.4. File Checksums	9
2.5. Project Overview	11
2.6. Project Timeline	11
3. Detailed Findings	12
3.1. Missing minimum deposit validation allows priority-queue spam	12
3.2. Missing validation of routeType compatibility with asset margin mode	14
3.3. Duplicate contracts can be added to the managedContracts array	16
4. Discussion	18
4.1. Trust assumptions	18

4.2.	ZkLighter is installed as an upgrade, not deployed fresh	18
5.	Patch Review	20
5.1.	Introduction of the L1CoreMessaging and interoperability system	20
5.2.	Changes in contract structure	22
6.	System Design	27
6.1.	Component: ZkLighter	27
6.2.	Component: L1CoreMessaging	29
6.3.	Component: L1InteropManager	32
6.4.	Component: Proxy and UpgradeGatekeeper	34
6.5.	Component: Verifiers	36
7.	Assessment Results	38
7.1.	Disclaimer	38

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Lighter from May 26th to June 3rd, 2026. During this engagement, Zellic reviewed Lighter (EVM)'s code for security vulnerabilities, design issues, and general weaknesses in security posture.

We were asked to review a minor patch to Lighter (EVM), based on the previous version, which refactored some contract structures and introduced a few new contracts.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the existing functionality preserved correctly under the new contract structure?
 - For the checks that were moved off chain into the circuits, does their removal from the on-chain contracts leave any gap in validation or access control?
 - Is access control correctly enforced across the contracts introduced by the refactor (the only-Core, only-governance, and only-interop-manager paths)?
 - Does the migration upgrade preserve the existing proxy storage layout and state?
 - Can funds become stuck, lost, or double-spent as a result of the refactor?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody
- Off-chain system components (sequencer, prover, indexer, zero-knowledge circuits)

We assumed these off-chain components are functioning as intended and focused exclusively on the on-chain smart contract security.

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

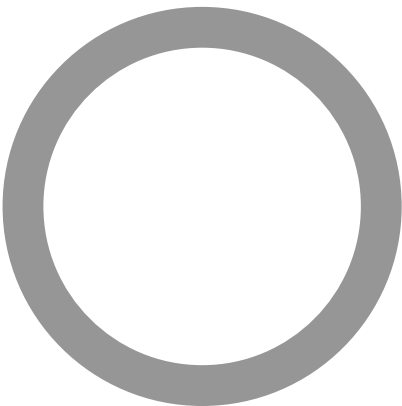
1.4. Results

During our assessment on the scoped Lighter (EVM) contracts, we discovered three findings, all of which were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Lighter in the Discussion section ([4. 7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	3



2. Introduction

2.1. About Lighter (EVM)

Lighter contributed the following description of Lighter (EVM):

Lighter is a decentralized trading platform built for unmatched security and scale. It's the first exchange to offer verifiable order matching and liquidations while delivering best-in-class performance on par with traditional exchanges.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Lighter (EVM) Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only changes between 6edc3ff...c768c3e
Repository	https://github.com/elliotttech/lighter-contracts-internal.git
Version	c768c3eb49f4fc031a7d4b03b50bf37a9e4a69f3
Programs	contracts/**/* .sol Excluded contracts/tests/* .sol

2.4. File Checksums

This assessment covered Lighter (EVM) at commit hash c768c3eb49f4fc031a7d4b03b50bf37a9e4a69f3. It was conducted as a differential review of the changes introduced since Zellic's previous Lighter (EVM) assessment, delivered in January 2026; only the modifications made relative to that prior baseline (commit 6edc3ff), together with the newly added contracts, were within the scope of this engagement.

The SHA-256 checksums of the audited source files are listed below. File paths are relative to the contracts/ directory.

File	SHA-256
AdditionalZkLighter.sol	4460901c409b54c7e5167e13489489d280504e8c53c27e4f2434ab5523632a0c
Config.sol	8bfa7bab14ceb0a9993311034131411872ba261e4aeacdf6fd8c59d91c0fb655
Constants.sol	e6bfc109199d685e38f7479796d608a101953df76371d73286b8a611548272c9
DeployFactory.sol	a3529e2342507a22d4d3b298b5019c800e9d920aa0b4e6cfe45adfb46a46bade
DesertVerifier.sol	ff6fea666b7e50ad55890bb01ad169a78ed8076cddac73ea7f4c28e570f96752

File	SHA-256
ExtendableStorage.sol	fe2d9eb1ea92fc8e144c77a91f3a24d69a2f5eee4b461c73cca2d2b5d8d15cd7
Governance.sol	d0f6235ff3ebac12ceb0d226a8e60df24aaabd15edfd7a653a9383f4b1ca68ff
interop/L1InteropManager.sol	3b0090ff818084e89cb53250910c3dd48cdcb3b362754b048060ff1465735d9
interop/TokenInteropLib.sol	5e642e07ede8c603f35a262ca48e1be2b791e82f5c4db7a80302539a9d8cda50
lib/Bytes.sol	8312145c6db2cc32c89e7b19685c91fb5d22c05a89fe05b3f32cf63a8b681788
lib/TxTypes.sol	18a21131c59779d875f1392de56e909aa2a99454e3fb02d0e9a07e182269d730
messaging/L1CoreMessaging.sol	1c9e10514fdbb2ba13f8a922eab4058472549f3f9571f83bc58c37a0a0ae260f
proxy/Ownable.sol	9af0bf6ff66d5e584949f1b7593c989262c9048e984dd2f428a4d62bb28c0b7b
proxy/Proxy.sol	fe5282428889e803e82db004b930315521ca4e98d2973b792e9b0b533dbcceee
proxy/UpgradeGatekeeper.sol	99ed0c6a01132938ad2257da93ff9f2925d7ce22a0f61a89a8626cb83f37c6f6
Storage.sol	922990e3e19d7db136c416cfaa5440dee6a2cdda88dab6e3f798ed9a839f639d
UpgradeableMaster.sol	487d58284ab0ea3091fd204ee87b68ed428ebf4a58b7d1f606da4458953d9d1e
ZkLighter.sol	b4f25ee5a6b225da5d77acd270d05db5d4e1f89fd1e6698f3b836cbe67b58104
ZkLighterStateRootUpgradeVerifier.sol	2efddddd20152ca4cd86a25d3e9054f842452a701b3561cce0fba872e4c138b5d
ZkLighterVerifier.sol	f6fe0e83cc746cf634f18e2253fbfd0e47fa727f1bd98c338f5fa11a0c80ee22

The following script was used to compute these checksums, should the reader wish to reproduce them (run from the root of the repository at the audited commit):

```
#!/usr/bin/env zsh
files=(
  contracts/AdditionalZkLighter.sol
  contracts/Config.sol
  contracts/Constants.sol
  contracts/DeployFactory.sol
  contracts/DesertVerifier.sol
  contracts/ExtendableStorage.sol
  contracts/Governance.sol
  contracts/interop/L1InteropManager.sol
  contracts/interop/TokenInteropLib.sol
  contracts/lib/Bytes.sol
  contracts/lib/TxTypes.sol
  contracts/messaging/L1CoreMessaging.sol
  contracts/proxy/Ownable.sol
  contracts/proxy/Proxy.sol
  contracts/proxy/UpgradeGatekeeper.sol
  contracts/Storage.sol
  contracts/UpgradeableMaster.sol
  contracts/ZkLighter.sol
  contracts/ZkLighterStateRootUpgradeVerifier.sol
  contracts/ZkLighterVerifier.sol
)
shasum -a 256 $files | awk '{printf "%-48s - %s\n", $2, $1}'
```

2.5. Project Overview

Zellic was contracted to perform a security assessment for a total of two person-weeks. The assessment was conducted by two consultants over the course of one calendar week.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Filipe Alves
✈ Engineer
filipe@zellic.io ↗

Kritsada Dechawattana
✈ Engineer
kritsada@zellic.io ↗

2.6. Project Timeline

The key dates of the engagement are detailed below.

May 26, 2026 Start of primary review period

May 27, 2026 Kick-off call

June 3, 2026 End of primary review period

3. Detailed Findings

3.1. Missing minimum deposit validation allows priority-queue spam

Target	L1InteropManager		
Category	Business Logic	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The `L1InteropManager::sendToCore()` function accepts deposits of any size (as long as they are multiples of `tickSize`) without enforcing minimum deposit thresholds. While the codebase defines an `ETH_MIN_DEPOSIT_TICKS` constant with a value of 100_000 ticks (equivalent to 0.001 ETH), this constant is never used in validation logic.

The deposit validation in `sendToCore()` only checks for nonzero amounts and tick alignment:

```
function sendToCore(uint16 _assetIndex, address to, uint256 amount,
    TxTypes.RouteType routeType) external payable nonReentrant {
    L1InteropManagerStorage storage $ = _getL1InteropManagerStorage();
    CoreAssetL1Config memory cfg = $.assetConfigs[_assetIndex];

    // Validation checks
    if (!isActiveOnL1(_assetIndex))
        revert TokenInteropLib.TokenInteropLib_AssetNotActive();
    if (to == address(0))
        revert TokenInteropLib.TokenInteropLib_RecipientAddressInvalid();
    if (amount == 0 || amount % cfg.tickSize != 0)
        revert TokenInteropLib.TokenInteropLib_InvalidDepositAmount();
    // Missing: minimum deposit check

    uint64 baseAmount = TokenInteropLib.toBaseAmount({amount: amount,
        tickSize: cfg.tickSize});
    L1_CORE_MESSAGING.registerDeposit{value: msg.value}(msg.sender, to,
        _assetIndex, routeType, baseAmount);
}
```

In contrast, the L2-side `AssetConfig` struct in the `ExtendableStorage` contract properly includes `minDepositTicks`, but this field is not synchronized with the L1 interop layer.

Note: The Lighter team confirmed that minimum-deposit validation is enforced at the circuit level rather than in the L1 contracts, so deposits below the configured minimum are rejected during proof generation. This finding is therefore reported as a defense-in-depth observation; enforcing the check at the L1 entry point would additionally prevent invalid deposits from ever entering the priority queue.

Impact

Attackers can spam the priority queue with dust deposits (e.g., 1 wei for ETH, or minimal token amounts for ERC-20s) that force validators to spend more gas processing them than the deposit value. This creates a griefing attack that delays legitimate deposits and bloats storage while the attacker incurs minimal cost.

Recommendations

Use the defined `ETH_MIN_DEPOSIT_TICKS` constant in the deposit flow. This ensures all assets have economically meaningful minimum deposits and protects the protocol from spam attacks. Alternatively, if the minimum-deposit check is intended to live solely at the circuit level, remove the unused `ETH_MIN_DEPOSIT_TICKS` constant, as a defined-but-unused threshold is misleading and implies a check that the L1 deposit flow does not perform.

Remediation

This issue has been acknowledged by Lighter.

Lighter provided the following response to this finding:

We will keep this validation in the circuits to allow separate handling for divergences that can occur between EVM deposits and core deposits, relying on Ethereum gas cost working against spams.

3.2. Missing validation of routeType compatibility with asset margin mode

Target	L1InteropManager		
Category	Business Logic	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The `sendToCore()` function in the `L1InteropManager` contract allows users to deposit assets to either the perps or spot route without validating whether the asset's margin mode is compatible with the chosen route.

Note: The Lighter team confirmed that `routeType`-to-margin-mode compatibility is validated at the circuit level rather than in the L1 contracts, so incompatible deposits are caught during proof generation. This finding is therefore reported as a defense-in-depth observation; validating the route at the L1 entry point would surface the error before the deposit is submitted.

Impact

Users who deposit non-margin assets (e.g., spot-only tokens with `marginMode = Disabled`) to `RouteType.Perps` will have their funds credited to the perpetuals account pool where they cannot be used as collateral. While funds are not permanently lost (users can withdraw and redeposit), users must withdraw from the perps pool back to L1 and then deposit again to the correct route to recover. This requires an additional L1 transaction and a time delay.

Recommendations

Add validation in `L1CoreMessaging.registerDeposit()` to check that the given `routeType` matches the asset's margin mode.

Remediation

This issue has been acknowledged by Lighter.

Lighter provided the following response to this finding:

We will keep this validation in the circuits to allow separate handling for divergences that can occur between EVM deposits and core deposits, relying on Ethereum gas cost working against spams.

3.3. Duplicate contracts can be added to the managedContracts array

Target	UpgradeGatekeeper		
Category	Business Logic	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The addUpgradeable function in the UpgradeGatekeeper contract allows adding contract addresses to the managedContracts array without verifying whether the address already exists. There is no duplicate check before pushing the new address:

```
function addUpgradeable(address addr) external {
    requireMaster(msg.sender);
    require(upgradeStatus == UpgradeStatus.Idle, "apc11");

    managedContracts.push(IUpgradeable(addr));
    emit NewUpgradeable(versionId, addr);
}
```

When finishUpgrade is called, it iterates through the entire managedContracts array and calls upgradeTarget() on each entry. If a contract appears multiple times in the array, its upgrade function will be invoked multiple times during a single upgrade operation:

```
function finishUpgrade(bytes[] calldata targetsUpgradeParameters) external {
    [...]
    for (uint64 i = 0; i < managedContracts.length; i++) {
        address newTarget = nextTargets[i];
        if (newTarget != address(0)) {
            managedContracts[i].upgradeTarget(newTarget,
            targetsUpgradeParameters[i]);
        }
    }
    [...]
}
```


Impact

If the same proxy is registered twice, `finishUpgrade` calls `upgradeTarget()` on it once per duplicate slot, running the implementation's `upgrade()` hook twice. In the code at the time of writing, this is benign, as every managed contract's `upgrade()` hook is effectively a no-op. However, these hooks are rewritten for each upgrade to carry migration logic, and a future nonidempotent hook invoked twice could corrupt storage or revert; since `finishUpgrade` is atomic, a revert would block the entire batched upgrade for all managed contracts.

The condition is heavily constrained: it requires the governor to call `addUpgradeable` twice with the same address, and since there is no `removeUpgradeable`, the duplicate is permanent. It can, however, be neutralized on each upgrade by setting the duplicate slot's `nextTarget` to `address(0)`, which `finishUpgrade` skips.

Recommendations

Add a duplicate check in the `addUpgradeable` function to prevent the same address from being added multiple times.

Remediation

This issue has been acknowledged by Lighter.

Lighter provided the following response to this finding:

We'll rely on upgradeable contracts lacking the upgrade function, and pay special attention to not register an upgradeable twice if we ever need to add a non-idempotent upgrade function.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Trust assumptions

Two of the system's most powerful on-chain operations perform no semantic validation of their own and rely on trust assumptions about the off-chain Core circuit rather than enforcing correctness themselves:

1. In `AdditionalZkLighter`, `executeL1Call` dispatches an arbitrary `to.call{value}(data)` for any call whose hash has reached the Ready state. The arguments are bound by `_computeL1CallHash`, but a hash only becomes Ready through a validity-proven batch's L1Call pubdata (or the desert path). The on-chain code never inspects what the call does, so a circuit bug, or a divergence between the circuit and the on-chain L1Call encoding, could authorize releasing the contract's ETH or invoking arbitrary targets. The transient `L2Sender` that `L1InteropManager.receiveFromCore` authenticates against is likewise only as trustworthy as the Core-attested origin committed into the call.
2. The verifier contracts are generated artifacts whose soundness rests on their verification keys matching the proving circuits, a correspondence that cannot be checked from the Solidity alone.

These are inherent properties of a validity-rollup design rather than defects, but they delineate the trust boundary; the safety of L1 fund custody reduces to the correctness of the off-chain circuits and the proving and verification key setup, which were outside the scope of this review.

4.2. ZkLighter is installed as an upgrade, not deployed fresh

The audited ZkLighter implementation does not implement an `initialize(bytes)` function; it exposes only a constructor (which sets `immutables` and calls `_disableInitializers()`) and an `upgrade(bytes)` entry point. As a result, deploying it fresh through the Proxy path would revert; the Proxy constructor `delegatecalls` `initialize(bytes)` on the implementation, and the missing function triggers the `uin11` ("target initialization failed") revert.

```
constructor(address target, bytes memory targetInitializationParameters)
    Ownable(msg.sender) {
    setTarget(target);
    (bool initializationSuccess, ) = getTarget().delegatecall(
```

```
        abi.encodeWithSignature("initialize(bytes)",
        targetInitializationParameters)
    );
    require(initializationSuccess, "uin11"); // uin11 - target initialization
    failed
}
```

This is intentional. The implementation under review is not meant to be deployed fresh; it is installed as an upgrade onto the existing, already initialized ZkLighter proxy through the UpgradeGatekeeper → Proxy.upgradeTarget → upgrade(bytes) path. The proxy's initialize runs once at its original deployment and is never invoked again, so the dropped initialize and the new upgrade are the migration counterparts of one another.

The DeployFactory contract, which still calls initialize during genesis, has not changed since the initial commit and is not used for production deployment. Lighter confirmed that DeployFactory is only used for the initial (already completed) deployment and that local development injects an initialize function into ZkLighter through a separate build-time source file for contract-size reasons. We note this so the missing initialize is not mistaken for a deployment-blocking defect; it is nonetheless worth confirming that DeployFactory remains excluded from any production deployment flow.

5. Patch Review

The purpose of this section is to document the exact commit diffs of the codebase that were considered in scope for this audit.

As requested by Lighter, we focused on the changes made between commit [6edc3ff2](#) and the commit [c768c3eb](#).

5.1. Introduction of the L1CoreMessaging and interoperability system

The update introduced several new contracts and libraries to support a modular interoperability architecture, separating concerns between the ZkLighter rollup contract and a new messaging layer for Core protocol interactions.

Contract: L1CoreMessaging

The L1CoreMessaging contract is a messaging module that handles priority request enqueueing for governance and user actions. It contains the following functions:

Governance functions

- The `setSystemConfig` function, which configures system parameters
- The `setTreasury` function, which updates the treasury address
- The `setInsuranceFundOperator` function, which updates the insurance-fund-operator address
- The `updateAsset` function, which updates existing asset parameters
- The `createMarket` function, which creates new markets with metadata
- The `updateMarket` function, which updates existing market parameters and validates based on market type

User functions

- The `withdraw` function, which enqueues withdrawal requests by reading the master account index and validating asset registration
- The `changePubKey` function, which allows users to change their Lighter public key for an account API key slot
- The `cancelAllOrders` function, which cancels all orders for a Lighter account
- The `createOrder` function, which creates orders for Lighter accounts with validation of parameters
- The `burnShares` function, which burns shares of an account in a public pool

Interop manager functions

- The `registerL1NativeAssetOnCore` function, which sends L1-native asset-registration priority requests to Core (only callable by InteropManager)
- The `registerCoreNativeAssetOnCore` function, which sends Core-native asset-registration priority requests to Core (only callable by InteropManager)
- The `registerDeposit` function, which handles deposit registration from the

InteropManager, managing account index assignment and forwarding deposits to ZkLighter

Helper functions

- The `getAccountIndexFromAddress` function, which resolves L1 addresses to account indexes
- The `getSystemAddress` function, which resolves system account indexes to their current addresses

Contract: L1InteropManager

The L1InteropManager contract manages the registration and bridging of assets between L1 and the Core protocol. It contains the following functions:

Governance functions

- The `registerL1NativeAssetConfig` function, which stores L1 configuration for an L1-native asset without sending to Core
- The `registerCoreNativeAssetConfig` function, which stores L1 configuration for a Core-native asset without sending to Core
- The `registerL1NativeAssetOnCore` function, which sends an L1-native asset-registration priority request to Core
- The `registerCoreNativeAssetOnCore` function, which sends a Core-native asset-registration priority request to Core
- The `setCoreTokenAdmin` function, which updates the Core token admin for an asset

User functions

- The `sendToCore` function, which allows users to deposit assets to Core by transferring tokens to the contract and registering the deposit with L1CoreMessaging

System functions

- The `receiveFromCore` function, which handles withdrawals from Core to L1 (callable only by ZkLighter when the L2 sender is the Core system)

View functions

- The `isRegisteredOnL1` function, which checks if an asset config has been registered on L1
- The `isActiveOnL1` function, which checks if the asset has an L1 token address set
- The `getAssetConfig` function, which returns the full asset configuration
- The `l1TokenToAssetIndex` function, which maps L1 token addresses to asset indexes

AdditionalZkLighter contract refactoring

The AdditionalZkLighter contract was significantly refactored to delegate most functionality to the new L1CoreMessaging contract:

- All user-facing functions were removed: `deposit`, `depositBatch`, `changePubKey`, `registerAsset`, `updateAsset`, `createMarket`, `updateMarket`, `withdraw`, `createOrder`, `cancelAllOrders`, and `burnShares`.
- The internal `_deposit` helper function was removed.
- Numerous error definitions related to validation were removed and moved to L1CoreMessaging.
- New errors were added: `AdditionalZkLighter_OnlyCoreMessaging`, `AdditionalZkLighter_L1CallHashNotReady`, `AdditionalZkLighter_L2SenderNotZero`, `AdditionalZkLighter_CannotExecuteToSelf`, `AdditionalZkLighter_L1CallFailed`, and `AdditionalZkLighter_InvalidBatch`.
- The `onlyCoreMessaging` modifier was added to restrict certain operations to the CoreMessaging contract.
- The `executeL1Call` and `l2Sender` functions were added, implementing the new pull-based L1 call flow (described below).

L1 call execution flow

The update introduces a pull-based mechanism for executing L1 calls (such as Core-to-L1 withdrawals), replacing the previous direct withdrawal path:

- When a batch is committed, each `L1Call` pubdata entry persists its `l1CallHash` to the new `l1CallStatus` mapping with status `Ready` (via `_persistL1CallHash`, which reverts on a preexisting hash). The desert-mode path persists hashes equivalently through `_persistDesertL1CallHash`.
- The `AdditionalZkLighter.executeL1Call(_l2Sender, to, value, msgNonce, data)` function is permissionless and recomputes the hash, requires its status to be `Ready`, marks it `Fulfilled`, and dispatches the call (`to.call{value}(data)`). It guards against reentrancy by reverting if the transient `l2Sender` slot is already set, and rejects calls targeting the contract itself.
- During the call, the originating L2 sender is held in transient storage (EIP-1153) and exposed via the new `l2Sender()` getter. This is how `L1InteropManager.receiveFromCore` authenticates its caller: the `onlyCoreSystem` modifier requires `ZkLighter.l2Sender()` to equal `LIGHTER_CORE_SYSTEM_ADDR`.

IZkLighter interface refactoring

The IZkLighter interface was refactored to remove functions that moved to L1CoreMessaging:

- The following errors were removed: `ZkLighter_CannotBeInitialisedByImpl`, `ZkLighter_InvalidInitializeParameters`, `ZkLighter_OnlyZkLighter`, `ZkLighter_TreasuryCannotBeZero`, `ZkLighter_TreasuryCannotBeInUse`, `ZkLighter_InsuranceFundOperatorCannotBeZero`, `ZkLighter_InsuranceFundOperatorCannotBeInUse`, and `ZkLighter_InvalidAssetConfigParams`.
- New errors were added: `ZkLighter_DesertRecipientMismatch`, `ZkLighter_L1CallHashAlreadyExists`, and `ZkLighter_InvalidMigrationTransfer`.
- Function signatures were removed: `deposit`, `depositBatch`, `changePubKey`, `registerAssetConfig`, `updateAssetConfig`, `registerAsset`, `updateAsset`, `createMarket`, `updateMarket`, `cancelAllOrders`, `withdraw`, `createOrder`, and `burnShares`.
- The `performDesert` function signature was updated to include `_masterAccountIndex` and `_to` parameters.

Storage contract refactoring

The Storage contract was refactored to support the new architecture:

- The `priorityRequests` mapping type changed from `uint64` to `uint256` for the key.
- The `additionalZkLighter` field type changed from `AdditionalZkLighter` to `address`.
- The `lastAccountIndex` and `addressToAccountIndex` fields were made internal (renamed to `_lastAccountIndex` and `_addressToAccountIndex`).
- The new internal helper `_hashPubDataPrefixed` was added for efficient public data hashing using assembly.
- The new internal helper `_computeL1CallHash` was added for computing L1 call hashes using assembly.
- The `getAccountIndexFromAddress` internal function now uses the internal `_addressToAccountIndex` mapping.

TxTypes library refactoring

The TxTypes library was extensively refactored to use individual parameters instead of struct parameters:

- New priority request types were added: `PriorityPubDataTypeL1UnstakeAssets`, `PriorityPubDataTypeL1SetSystemConfig`, and `PriorityPubDataTypeL1RegisterCoreNativeAsset`.
- The `L1Call` variant was added (`OnChainPubDataType` enum change).
- Structs were removed: `Deposit`, `L1Withdraw`, `Withdraw`, `USDCWithdraw`, `CreateOrder`, `BurnShares`, `CancelAllOrders`, and `ChangePubKey`. These were replaced with individual parameter functions.
- As part of deserialization refactoring, the `readDepositForDesertMode` function was replaced with `readDepositFromCalldata`, which uses assembly for efficient calldata

reading. Additionally, the `readWithdrawOnChainLog` and `readUSDWithdrawOnChainLog` functions were removed.

- As part of `RegisterAsset` struct changes, the `extensionMultiplier` field was removed from the struct. Also, the `loanToValue`, `liquidationThreshold`, `liquidationFactor`, `liquidationFee`, and `indexPriceDivider` fields were added.

Staking pool support

The update introduces plumbing for a protocol-level staking pool, configured alongside the existing liquidity pool through the new system-config flow:

- The `Config` contract adds `MIN_STAKING_SHARES_TO_MINT_OR_BURN` (1) and `MAX_STAKING_SHARES_TO_MINT_OR_BURN` ($2^{60} - 1$), bounding the staking share mint/burn amounts.
- The `TxTypes.SetSystemConfig` struct (consumed by `L1CoreMessaging.setSystemConfig`) carries `stakingPoolIndex` and `stakingPoolLockupPeriod`, mirroring the `liquidityPoolIndex` and `liquidityPoolCooldownPeriod` fields. The `setSystemConfig` function bounds the lockup period by `MAX_CONFIG_PERIOD` and the pool index by `MAX_ACCOUNT_INDEX`.
- A new priority-request type, `PriorityPubDataTypeL1UnstakeAssets` (51), is reserved for the unstaking flow.

Note that, in the reviewed range, `PriorityPubDataTypeL1UnstakeAssets` is defined but not yet wired to any serializer or user-facing entry point; the L1 layer exposes only `burnShares` for public-pool exits. The staking and unstaking flows therefore appear only partially implemented at the contract layer in scope.

Config contract changes

The `Config` contract was updated to include additional configuration constants for the new system, including the staking share bounds described above.

ExtendableStorage changes

The `ExtendableStorage` contract gained storage to support the messaging and L1-call architecture, including

- a `coreMessaging` address field referencing the `L1CoreMessaging` contract,
- an `L1CallStatus` enum (`Nonexistent`, `Ready`, `Fulfilled`) and an `l1CallStatus` mapping that tracks the status of L1 call hashes consumed by `executeL1Call`, and
- a `desertModeNonce` field, a monotonically decreasing nonce used to derive desert-mode L1 call hashes.

Verifier contract changes

The verifier contracts were promoted from non-audited placeholders to generated production verifiers. Most significantly, ZkLighterVerifier was a nonfunctional stub in the base commit and is now a real generated PLONK verifier, adding BSB22 custom-gate commitment support (new logic in the proof-size computation and commitment hashing). Both DesertVerifier and ZkLighterStateRootUpgradeVerifier had their verification keys regenerated. As with any generated verifier, soundness rests on the keys matching the proving circuits and on the generator output itself – neither of which is verifiable from the Solidity alone.

IEvents interface changes

The IEvents interface was updated with new events emitted by the L1CoreMessaging and L1InteropManager contracts, including events for deposits, withdrawals, asset registration, market creation, and interoperability operations.

6. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

6.1. Component: ZkLighter

Description

The ZkLighter contract is the L1 settlement anchor of the Lighter rollup and the only contract that mutates the canonical rollup state on L1. It is deployed behind a Proxy, and most of its logic is delegated to the AdditionalZkLighter contract through a delegatecall in order to fit the contract-size limit.

ZkLighter is responsible for the following:

- **Driving the batch life cycle.** Batches are committed (`commitBatch`), validity-proven (`verifyBatch`), and applied (`executeBatches`), with the ability to roll back unexecuted batches (`revertBatches`). Each committed batch carries a blob-based public-data commitment and is bound to a ZK validity proof.
- **Holding native ETH custody for the exchange.** ERC-20 custody lives in the `L1InteropManager` contract instead.
- **Maintaining the priority-request queue.** Requests enqueued by `L1CoreMessaging` are stored as a rolling prefix-hash chain and later consumed by committed batches.
- **Dispatching outbound L1 calls.** State transitions proven on Core (the Lighter L2) that must take effect on L1 (for example, releasing funds) are recorded as `l1CallHash` entries during batch execution (`_persistL1CallHash`) and later executed permissionlessly through `executeL1Call`. A transient-storage slot carries the authenticated Core-side origin (`l2Sender`) for the duration of the external call.
- **Implementing desert (escape-hatch) mode.** When the operator stalls, an account can exit directly against a desert proof (`performDesert`), and pending deposits can be reclaimed (`cancelOutstandingDepositsForDesertMode`).

The contract was migrated from a standalone rollup implementation; its `initialize` function was removed (the live proxy is already initialized), and the new implementation is installed through the proxy upgrade path. The constructor records an immutable `MIGRATION_TARGET`.

Invariants

- The batch counters advance monotonically and in order: a batch can be verified only after it is committed and executed only after it is verified (`executedBatchesCount <= verifiedBatchesCount <= committedBatchesCount`).
- A batch can only be acted on if it matches its stored hash (`storedBatchHashes`) and can

only be executed if its number is within `verifiedBatchesCount` and it is the next in the on-chain execution queue.

- The priority-request queue is an append-only prefix-hash chain; a committed batch must match the chain at the committed count (`prefixPriorityRequestHash`).
- Each `l1CallHash` transitions only `Nonexistent` -> `Ready` -> `Fulfilled`. It can never be persisted twice (`_persistL1CallHash` reverts otherwise) nor executed twice (status is set to `Fulfilled` before the external call).
- An L1 call's effect is bound to its hash; `executeL1Call` recomputes the hash from its arguments and requires the status to be `Ready`.
- The `executeL1Call` function is non-reentrant for a nonzero `l2Sender` (transient-slot guard) and cannot target the proxy itself.
- Desert mode is one-way: once `desertMode` is set, it cannot be cleared, and all batch operations are disabled while it is active (`onlyActive`).
- Each `(asset, account)` pair can perform a desert exit at most once (`accountPerformedDesertForAsset`), and desert funds are only released to an address that resolves to the proven master account index.

Test coverage

Cases covered

- `commitBatch`: Happy path (with and without priority and on-chain operations) and validation reverts (inactive verifier, invalid pubdata commitments, invalid pubdata mode, nonincreasing block number, invalid batch size, nonincreasing timestamp, stored-batch mismatch, priority-prefix-hash mismatch).
- `verifyBatch`: Happy path (with and without priority and on-chain operations) and reverts (inactive verifier, invalid proof, invalid batch).
- `executeBatches`: Happy paths (single batch, multiple batches, called by a nonvalidator), `L1Call` hash persistence, and reverts (length mismatch, more batches than pending, nonverified batch, incorrect batch data, incorrect pubdata, incorrect pubdata type, incorrect pubdata length, skipping the execution queue).
- `cancelOutstandingDepositsForDesertMode`: One happy-path test.

Cases not covered

- `executeL1Call`: It is never invoked by the tests; the external dispatch, the transient `l2Sender` reentrancy guard, the `to == address(this)` check, the value sourcing, and the `Fulfilled` transition are untested (only the `Ready` persistence side, via `executeBatches`, is exercised).
- `performDesert`: The desert exit with proof verification, the recipient/master-index binding, and `_persistDesertL1CallHash` are untested.
- `revertBatches`: There is no test.
- `updateStateRoot` (state-root-upgrade verifier path and one-shot reset): There is no test.
- `upgrade` (migration wiring and commitment guard) and `transferTokens` (migration

transfer): There is no test.

- **11CallHash**: A duplicate 11CallHash appearing in a committed batch (the revert-on-duplicate liveness path) is not tested.

Attack surface

The contract exposes several permissionless entry points, so most of its surface is reachable by any caller.

- **Batch life cycle (commitBatch, verifyBatch, executeBatches, revertBatches)**. Committing and verifying are gated to active validators (`governance.isActiveValidator`), but execution is permissionless. An attacker controls the batch and on-chain operation data passed to these functions; integrity is enforced by matching against the stored batch hash, the verified-batch count, the priority-request prefix-hash chain, and the ZK validity proof. The main risk is supplying data that passes the on-chain structural checks but diverges from what the proof attests, which is mitigated only if the circuit and the on-chain encoding agree.
- **executeL1Call**. This is fully permissionless. The caller supplies (`l2Sender`, `to`, `value`, `msgNonce`, `data`); these are bound by recomputing the call hash and requiring it to be in the Ready state, so the arguments cannot be tampered with. The relevant attacker-reachable concerns are reentrancy (mitigated by the transient `l2Sender` lock, though that lock is ineffective when `l2Sender` is the zero address), self-calls (blocked by `to != address(this)`), and the fact that the released value is drawn from the contract's own balance.
- **performDesert / cancelOutstandingDepositsForDesertMode**. These are reachable by anyone once desert mode is active. The attacker controls the account index, master account index, asset, amount, recipient, and proof. Misdirection is prevented by requiring the desert proof to verify against the exit commitment and by requiring the recipient to resolve to the proven master account index; double exits are blocked per (`asset`, `account`). The deposit-cancellation path is constrained by the priority-request prefix-hash chain.
- **11CallHash uniqueness**. Because `_persistL1CallHash` reverts on a preexisting hash, an attacker (or a faulty Core sequencer) that can cause a duplicate hash to appear in a committed batch can permanently block batch execution.

6.2. Component: L1CoreMessaging

Description

The L1CoreMessaging contract is the inbound messaging gateway of the system. It receives user and governance actions on L1 and converts them into priority requests that the Core protocol (L2) sequencer later processes. It is deployed behind a Proxy and holds immutable references to the ZkLighter and L1InteropManager contracts.

L1CoreMessaging is responsible for the following:

- **Enqueuing user actions.** These include `withdraw`, `changePubKey`, `cancelAllOrders`, `createOrder`, and `burnShares` (the latter targeting public/liquidity-pool shares).
- **Enqueuing governance actions.** These include `setSystemConfig`, `setTreasury`, `setInsuranceFundOperator`, `createMarket`, `updateMarket`, `updateAsset`, and the asset-registration relays (`registerL1NativeAssetOnCore`, `registerCoreNativeAssetOnCore`).
- **Owning the L1 account registry.** It maps L1 addresses to master account indexes and assigns new indexes on deposit (`registerDeposit`). The registry is seeded at migration time from the existing ZkLighter `lastAccountIndex`, preserving account-index continuity.
- **Handling deposits forwarded by the L1InteropManager.** The `registerDeposit` function (callable only by the `L1InteropManager`) resolves or creates the destination account, builds the deposit public data, and forwards the request — and any attached ETH value — to ZkLighter via `addPriorityRequestExt`.

Each action is bound to the caller's master account index, which is derived from `msg.sender` and therefore cannot be spoofed. The contract performs only structural validation (asset registration, parameter bounds, route type); the actual ownership and balance checks are deferred to the Core circuit that consumes the priority requests.

Invariants

- Every enqueued action is bound to the caller's master account index, which is derived from `msg.sender` and cannot be spoofed; callers without a registered account (`NIL_ACCOUNT_INDEX`) are rejected.
- Account indexes are assigned monotonically (`++lastAccountIndex`) and never exceed `MAX_MASTER_ACCOUNT_INDEX`, and the counter is seeded at migration from the ZkLighter `lastAccountIndex` so indexes remain continuous.
- Only the `InteropManager` can call `registerDeposit` and the asset-registration relays (`onlyInteropManager`); only governance can call the governance functions (`onlyGovernance`).
- The `registerDeposit` function forwards exactly the received `msg.value` to ZkLighter.
- The treasury and insurance-fund operator must be nonzero, distinct from each other, and not collide with an already registered account.
- The contract performs only structural validation (asset registration, parameter bounds, route type); balance and ownership correctness are deferred to the Core circuit that consumes the priority requests.

Test coverage

Cases covered

- `createMarket` / `updateMarket` validation (perps and spot): Extensive revert cases (invalid market type/index, fees, margin fraction, minimum amounts, market limits, interest rate, funding clamps, open-interest limit, order-quote limit, status, length).

- Public-key validation: All-zeros rejection, field-element range check, little-endian reads, and byte-swap helpers.
- `updateAsset`: Reverts when the caller is not governor and when the asset is inactive.
- `registerL1NativeAssetOnCore`: Reverts when the caller is not the manager and forwards the expected metadata layout.
- `registerDeposit`: Reverts when the caller is not the manager and emits the deposit via the helper.
- `withdraw`: Uses the caller's account and reverts when the asset is inactive and when the amount is zero.
- `changePubKey`: Succeeds for a registered user.
- `setTreasury` / `setInsuranceFundOperator`: Success plus reverts (not governor, zero address, in-use account, equal to the counterpart).

Cases not covered

- `setSystemConfig` (system and staking configuration, fee-cap validation): There is no test.
- `burnShares`, `createOrder`, and `cancelAllOrders` happy paths and their validation: There are no tests.
- The new-account creation path inside `registerDeposit` (index assignment and `TooManyRegisteredAccounts`): Only the helper-emit path is exercised.
- `registerCoreNativeAssetOnCore`: There is no test.

Attack surface

The contract is the main user-facing entry point, so its user functions are permissionless and the attacker controls all of their arguments.

- **User functions** (`withdraw`, `changePubKey`, `cancelAllOrders`, `createOrder`, and `burnShares`). The caller supplies the target account index, asset, amounts, route type, and other parameters. The contract enforces only structural validation (asset registration, parameter bounds, route type, public-key format) and binds the request to the caller's master account index, which is derived from `msg.sender` and therefore cannot be spoofed. Ownership of the target account and balance sufficiency are not checked on L1; they are deferred to the Core circuit. An attacker can enqueue requests referencing accounts they do not own, but the circuit is expected to reject them. A key assumption is that a non-spoofable `masterAccountIndex` is sufficient for the circuit to authorize the action.
- **registerDeposit and asset-registration relays**. These are restricted to the `InteropManager` (`onlyInteropManager`); the deposit recipient and amount originate from `sendToCore`. The new-account branch increments `lastAccountIndex` and is bounded by `MAX_MASTER_ACCOUNT_INDEX`. The forwarded `msg.value` must match what the interop manager received.
- **Governance functions**. These are restricted to the governor (`onlyGovernance`); the attacker surface here is limited to a compromised or misconfigured governor.

- **Account registry.** The address-to-index mapping is attacker-influenced through deposits (an attacker can cause new indexes to be assigned by depositing to fresh addresses), which is the data later relied upon by desert-mode recipient resolution.

6.3. Component: L1InteropManager

Description

The L1InteropManager contract manages the registration and bridging of assets between L1 and the Core protocol. It is deployed behind a Proxy, uses ERC-7201 namespaced storage, and holds immutable references to the ZkLighter and L1CoreMessaging contracts. It is the custodian of ERC-20 deposits; native ETH custody remains in ZkLighter.

L1InteropManager is responsible for the following:

- **Inbound bridging (sendToCore).** It pulls ERC-20 tokens from the depositor (or accepts ETH), validates the amount against the asset's `tickSize`, converts it to a base amount, and registers the deposit through L1CoreMessaging.
- **Outbound bridging (receiveFromCore).** It releases ERC-20 to a recipient. It is callable only by ZkLighter while an `executeL1Call` is in progress whose authenticated origin (`l2Sender`) is the Core system address, ensuring the payout corresponds to a proven Core state transition.
- **Asset configuration.** Governance registers L1-native and Core-native asset configs (`registerL1NativeAssetConfig`, `registerCoreNativeAssetConfig`), relays registrations to Core, and updates the Core token admin (`setCoreTokenAdmin`). Each asset config records its canonical domain, L1 token address, tick size, and extension multiplier.

Asset state is tracked through two predicates: `isRegisteredOnL1` (a config exists for the asset) and `isActiveOnL1` (the asset has a nonzero L1 token address). Amount conversions on L1 use only the `tickSize` ($\text{amount} = \text{baseAmount} * \text{tickSize}$); the extension multiplier is forwarded to Core and applied in the Core representation. Reentrancy is guarded with a transient-storage lock.

Invariants

- The ERC-20 balance custodied by the manager corresponds to the net base amount bridged in and not yet released; `transferIn` reverts on any balance delta that does not match the requested amount (rejecting fee-on-transfer tokens).
- A deposit amount must be a nonzero multiple of the asset's `tickSize`, and the conversion is exact and reversible ($\text{baseAmount} = \text{amount} / \text{tickSize}$, $\text{amount} = \text{baseAmount} * \text{tickSize}$).
- ETH deposits require `msg.value == amount`; ERC-20 deposits require `msg.value == 0`.
- The `receiveFromCore` function is callable only by ZkLighter while an `executeL1Call` is in progress whose `l2Sender` is the Core system address — and never for the native asset.

- An asset index is registered at most once (`AssetAlreadyRegistered`); index 0 is reserved; the native asset (index 1) is fixed at initialization, and its admin is immutable.
- Both `sendToCore` and `receiveFromCore` operate only on assets that are active on L1 (`isActiveOnL1`).
- The transient-storage reentrancy guard prevents nested `sendToCore` calls.

Test coverage

Cases covered

- `setCoreTokenAdmin`: Reverts (ETH admin immutable, asset not active) and the success event.
- `receiveFromCore`: Reverts (caller not authorized, ETH not supported, asset not active).
- `registerL1NativeAssetConfig` / `registerCoreNativeAssetConfig`: Revert (asset index reserved, asset index too high, L1 token zero address, asset already registered, L1 token already registered).

Cases not covered

- The `sendToCore` function has no tests at all; the entire deposit happy path (token pull, tick-size validation, base-amount conversion, ETH-versus-ERC-20 branching, and forwarding to `L1CoreMessaging`) is untested.
- The `receiveFromCore` success path (the actual ERC-20 release) is untested; only the revert cases are covered.
- The success paths of `registerL1NativeAssetConfig` / `registerCoreNativeAssetConfig` and the on-Core registration relays are not covered.
- Consistency edge cases between `isRegisteredOnL1` and `isActiveOnL1` (for example, a core-native asset with no L1 token) are not covered.

Attack surface

This contract custodies ERC-20 deposits, so its deposit and withdrawal paths are the most value-sensitive surfaces.

- **`sendToCore`**. This is permissionless and payable. The caller controls the asset index, recipient, amount, and route type. The contract checks that the asset is active on L1, that the recipient is nonzero, and that the amount is a nonzero multiple of `tickSize`; for ETH, it requires `msg.value == amount`, and for ERC-20, it requires `msg.value == 0` and pulls tokens via `transferIn` (which rejects fee-on-transfer mismatches). No minimum-deposit threshold is enforced, so dust deposits are accepted; the route type is not validated against the asset's margin mode. The base-amount conversion (`amount / tickSize`) must fit in a `uint64`.
- **`receiveFromCore`**. This is the release path. It is restricted to `ZkLighter` while an `executeL1Call` whose `l2Sender` is the Core system address is in progress (`onlyCoreSystem`), so an external attacker cannot call it directly. The asset index,

recipient, and base amount originate from a proven Core withdrawal; the contract rejects the native asset and inactive assets and computes the released amount as `baseAmount * tickSize`.

- **Asset configuration.** This is restricted to governance. The attacker surface is limited to a compromised governor, but note that a core-native asset is registered with no L1 token and has no path to become active, and the `withdraw` entry point on `L1CoreMessaging` uses the weaker `isRegisteredOnL1` predicate while redemption uses `isActiveOnL1`.
- **Reentrancy.** The transient-storage lock guards `sendToCore`; `receiveFromCore` relies on its caller restriction and on the in-progress `executeL1Call` context rather than the local lock.

6.4. Component: Proxy and UpgradeGatekeeper

Description

All core contracts (`ZkLighter`, `Governance`, `ZkLighterVerifier`, `L1CoreMessaging`, and `L1InteropManager`) run behind a Proxy contract that stores the implementation address in the EIP-1967 implementation slot. The Proxy sets its initial implementation at construction (delegating to `initialize`) and changes it through `upgradeTarget` (delegating to `upgrade`); direct `initialize` and `upgrade` calls to the proxy are intercepted.

The `UpgradeGatekeeper` contract is the master of these proxies and governs implementation changes through a notice-period state machine:

- `addUpgradeable` registers a proxy under the gatekeeper's control.
- `startUpgrade` proposes new targets and starts the notice period.
- `startPreparation` advances to the preparation phase once the notice period elapses.
- `finishUpgrade` applies the new targets by calling `upgradeTarget` on each managed proxy.
- `cancelUpgrade` aborts an in-progress upgrade.

Upgrades are gated by a security council and are disabled while the rollup is in desert mode. In the EVM migration, the new `ZkLighter` implementation is installed onto the already live proxy through this upgrade path rather than through a fresh deployment, and the new implementation's upgrade function performs one-time wiring of the messaging and interop components (guarded by a hardcoded parameter commitment).

Invariants

- The implementation address can only be changed by the proxy master (the `UpgradeGatekeeper`) through `upgradeTarget`; direct `initialize` and `upgrade` calls on the proxy are intercepted and revert.
- Upgrades follow the state machine `Idle` -> `NoticePeriod` -> `Preparation` -> `finish`; preparation can only begin after the notice period elapses, and the new-target

list length must equal the managed-contract list length.

- Upgrades are disabled while the rollup is in desert mode (`isReadyForUpgrade`).
- The `ZkLighter` upgrade function can only execute through the proxy (not against the implementation address) and only with the parameter set whose hash matches the hardcoded commitment.

Test coverage

Cases covered

- The Hardhat Proxy.test.ts suite covers the following: target-address storage, `upgradeTarget` for the Governance and `ZkLighterVerifier` targets and its event, `delegatecall` fall-through, `initialize` / upgrade interception, and the `implementation-cannot-self-upgrade` guard.

Cases not covered

- The `ZkLighter` upgrade case is explicitly skipped (`it.skip('upgrade zkLighter')`), so the actual migration upgrade path (the commitment-guarded `upgrade(bytes)`) is untested.
- The `UpgradeGatekeeper` state machine (`startUpgrade`, `startPreparation`, `finishUpgrade`, `cancelUpgrade`), the notice-period timing, desert-mode gating, and the `duplicate-managedContracts` behavior are not unit tested.

Attack surface

Upgradability concentrates the most privileged operations in the system, so its attack surface is dominated by the trust placed in the proxy master.

- **Privileged entry points.** All of the following — `addUpgradeable`, `startUpgrade`, `startPreparation`, `finishUpgrade`, and `cancelUpgrade` — are restricted to the gatekeeper master (`requireMaster`). The attacker surface is therefore primarily a compromised or misconfigured master/security council; for an external attacker, the surface is limited to the interception stubs and the public view of pending targets.
- **Master-controlled inputs.** The master supplies the new target addresses and the per-target upgrade parameters. There is no on-chain check that a new target is a valid implementation, that the parameters are well-formed, or that an address is not added more than once, so an operational mistake can install a wrong target or duplicate a managed contract (causing it to be upgraded twice in one round). The `finishUpgrade` loop calls `upgradeTarget` on each managed proxy with master-provided data.
- **Notice period and desert mode.** The notice-period state machine and the desert-mode gate (`isReadyForUpgrade`) are the main timing and safety constraints; bypassing or missequencing them is only reachable by the master.
- **Migration upgrade.** The new `ZkLighter` `upgrade(bytes)` is reachable only through the proxy and is constrained to the single parameter set whose hash matches the

hardcoded commitment, which narrows the surface to whether that commitment encodes the intended addresses.

6.5. Component: Verifiers

Description

Three verifier contracts validate the zero-knowledge proofs the rollup relies on. Each exposes a `Verify` entry point and embeds a verification key as contract constants.

1. `ZkLighterVerifier` validates batch validity proofs, consumed by `ZkLighter.verifyBatch` to advance the verified state.
2. `DesertVerifier` validates escape-hatch exit proofs, consumed by `ZkLighter.performDesert` to authorize a desert-mode withdrawal against the frozen state root.
3. `ZkLighterStateRootUpgradeVerifier` validates the one-time state-root upgrade proof, consumed by `AdditionalZkLighter.updateStateRoot`; the verifier is reset to the zero address after a successful upgrade, so it can only be used once.

During this engagement, the verification keys were regenerated, and the batch circuit's domain size grew substantially (from 64 to roughly 33.5 million), indicating a materially different proving circuit. The contracts are machine generated; their correctness depends on the embedded verification key corresponding to the intended prover circuit, which is confirmed out of band rather than by reading the generated constants.

Invariants

- A batch is only marked verified if `ZkLighterVerifier.Verify` returns true over the batch's public inputs.
- A desert exit is only authorized if `DesertVerifier.Verify` succeeds over the exit commitment.
- The state-root upgrade verifier is single-use; it is reset to the zero address after a successful `updateStateRoot`, so the same upgrade cannot be replayed.

Test coverage

Cases covered

- Verifier integration is exercised indirectly through the batch tests, using a mock verifier (`ZkLighterVerifierTest`) that returns success or failure on demand. This confirms that the calling logic accepts on `true` and reverts on `false`, not the cryptographic verification itself.

Cases not covered

- No tests exercise the real (generated) verification keys against real proofs for any of the three verifiers.
- The `DesertVerifier` and `ZkLighterStateRootUpgradeVerifier` integration paths (`performDesert`, `updateStateRoot`) are not tested with the real verifiers.

Attack surface

The verifiers are pure proof-checking contracts, so the attacker-controlled input is the proof (and its public inputs) submitted to each `Verify` call.

- **Proof data.** An attacker can submit arbitrary proof bytes and public inputs through the callers (`verifyBatch`, `performDesert`, `updateStateRoot`). Soundness rests entirely on the verification algorithm and the embedded verification key — a forged proof must fail verification. The public inputs are derived on chain from committed state (batch commitment, exit commitment, state-root transition), so the attacker cannot freely choose them independently of the proof.
- **Verification key correctness.** The dominant risk is not attacker input but configuration: if the embedded verification key does not correspond to the intended prover circuit, the verifier could accept proofs for an unintended statement or reject valid ones. This is confirmed out of band rather than by the contracts themselves.
- **State-root upgrade single use.** The `ZkLighterStateRootUpgradeVerifier` is consumed once and then reset to the zero address by `updateStateRoot`; the surface includes ensuring it cannot be rearmed or replayed.

7. Assessment Results

During our assessment on the scoped Lighter (EVM) contracts, we discovered three findings, all of which were informational in nature.

7.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.